Python Programming
Unit 05 – Lecture 04 Notes
Polymorphism (Overriding and Operator Overloading)

Tofik Ali

February 20, 2026

Contents

1 Lecture Overview

Polymorphism means the same method call can behave differently based on the object. In Python, this happens naturally through method overriding. Python also supports operator overloading using special (dunder) methods.

2 Core Concepts

2.1 Method Overriding

If a subclass defines a method with the same name as the parent method, it overrides it. When you call the method, Python uses the object's actual class.

```
class A:
    def greet(self):
        print("Hello from A")
```

```python
class B(A):
    def greet(self):
        print("Hello from B")

b = B()
b.greet() # Hello from B
```

2.2 Operator Overloading

Operators map to special methods. Some examples:

- `+` → `__add__`
- `-` → `__sub__`
- `==` → `__eq__`
- `<` → `__lt__`
- `len(x)` → `__len__`
- `print(x)` → `__str__`

2.3 Point Example

```python
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):
        return Point(self.x + other.x, self.y + other.y)
```

This allows `p3 = p1 + p2`.

2.4 When to Use Operator Overloading

Good cases:

- mathematical objects (Point, Vector, Matrix)
- domain objects where `+` has a clear meaning

Bad cases:

- when the meaning is unclear or surprising (hurts readability)

3 Demo Walkthrough

File: `demo/polymorphism_operator_overloading_demo.py`
Observe:

- different shapes implement `area()`
- `Point` supports `+` and has a readable `__str__`

4 Interactive Checkpoints (with Solutions)

Checkpoint 1 Solution

Question: If parent and child define `describe()`, which runs for child object?
Answer: The child's method runs (overriding).

Checkpoint 2 Solution

Question: Which special method for +?
Answer: `__add__`

5 Practice Exercises (with Solutions)

Exercise 1: Add `__str__` to Point

Task: Print `Point(10,20)` for a point object.
Solution:

```
def __str__(self):
    return f"Point({self.x},{self.y})"
```

Exercise 2: Implement Subtraction

Task: Implement `p1 - p2`.
Solution:

```
def __sub__(self, other):
    return Point(self.x - other.x, self.y - other.y)
```

6 Exit Question (with Solution)

Question: special method for printing?
Answer: `__str__`

Appendix: Slide Deck Content (Reference)

The material below is a reference copy of the slide deck content. Exercise solutions are explained in the main notes where applicable.

Title Slide

Repository: `https://github.com/tali7c/Python-Programming`

Quick Links

Core Concepts Demo Interactive Summary

Agenda

- Core Concepts
- Demo
- Interactive
- Summary

Learning Outcomes

- Explain polymorphism in OOP
- Implement method overriding in subclasses
- Implement operator overloading using special methods
- Use `__str__` for readable object printing

Polymorphism (Idea)

- "Same interface, different behavior"
- Example: `area()` behaves differently for Circle and Rectangle

Method Overriding

- A child class provides its own implementation of a parent method
- Python chooses the method based on the object's actual class

Operator Overloading

- Define how operators work for your objects
- Examples:
  - `+` uses `__add__`
  - `==` uses `__eq__`
  - `len(obj)` uses `__len__`

Example: Point Addition

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):
        return Point(self.x + other.x, self.y + other.y)
```

Demo: Overriding + Operator Overloading

- File: `demo/polymorphism_operator_overloading_demo.py`
- Shows:
  - polymorphic `area()` calls
  - `Point + Point` using `__add__`

Checkpoint 1

Question: If a parent and child both define `describe()`, which one runs for a child object?

Checkpoint 2

Question: Which special method is used for operator `+`?

Think-Pair-Share

Discuss:

- Should every class implement operator overloading? When is it helpful vs confusing?

Key Takeaways

- Overriding enables polymorphism (same method name, different behavior)
- Operator overloading uses special methods like `__add__`
- Use overloading only when it improves readability and matches meaning

Exit Question

Name the special method used to control printing of an object using `print(obj)`.